

React Native Technical Assignment

Build an Android product catalog application using Expo, React Native, TypeScript, Redux Toolkit, and RTK Query.

Completion deadline: Submit the completed task within 48 hours of receiving this assignment.

Project

Create a mobile application named **Product Pulse**. The application must load product data from `https://dummyjson.com` and allow users to browse products, view details, save favorites, and schedule product reminders.

Required technology

- Expo with React Native and Expo Router.
- TypeScript with strict type checking enabled.
- Redux Toolkit and RTK Query for API requests and remote data state.
- AsyncStorage, or an equivalent Expo-compatible local store, for persisted favorites and reminder records.
- expo-notifications for local notifications.
- EAS Build, or an equivalent Expo-compatible process, to generate an installable Android APK.

What you need to build

- 1 Create a product listing screen using GET `/products?limit={n}&skip={n}`. Display the product image, title, price, rating, category, and stock status.
- 2 Add pagination or infinite scrolling, pull-to-refresh, search using GET `/products/search?q={query}`, and one category filter or sorting control.
- 3 Create a product detail screen at `/products/{id}` using GET `/products/{id}`. Show images, description, price, rating, category, stock, and favorite/reminder actions.
- 4 Add a Favorites screen. Users must be able to add and remove products, and favorites must remain available after the app is closed and reopened.
- 5 Use RTK Query for list, search, category, and detail requests. Implement typed endpoints and hooks, API middleware, caching, refresh, loading, background fetching, empty, and error states.

What you need to build - continued

- 1 Configure the custom deep-link scheme `productpulse`. The link `productpulse://products/{id}` must open the matching product detail screen.
- 2 Verify the deep link when the app is terminated, running in the background, and open in the foreground. Invalid IDs or unknown routes must show a safe error screen and must not crash the app.
- 3 On the product detail screen, allow the user to schedule a one-time local reminder. Request notification permission only after the user selects this action.
- 4 Create an Android notification channel, schedule the notification, store its returned notification ID, and provide a Reminders screen where users can view and cancel scheduled reminders.
- 5 Store the route `/products/{id}` in the notification data. Tapping the notification must open the correct product from foreground, background, and terminated states.
- 6 Generate a standalone Android APK that installs and runs without Expo Go or a local Metro development server.

Required application quality

- Use reusable, clearly named components and keep API, navigation, storage, and notification logic separated.
- Avoid any. Type API responses, Redux state, route parameters, storage records, and notification data.
- Validate product IDs and notification/deep-link data before navigating or requesting data.
- Handle loading, refresh, empty, not-found, permission-denied, offline-like, and retry states.
- Use a virtualized list, stable keys, responsive layouts, safe areas, Android back behavior, accessible labels, readable contrast, and practical touch targets.
- Remove event listeners during cleanup and prevent duplicate requests, reminders, and navigation events.

Testing

- Add unit tests for deep-link and notification route validation.
- Add component tests for API loading, success, and error behavior.
- Test favorites persistence and reminder scheduling/cancellation using mocked service boundaries.
- Manually verify deep links and notification taps in terminated, background, and foreground app states.

Submission

- Source-code repository with setup instructions and a clear project structure.
- README containing run, test, build, APK installation, deep-link test, and notification test commands.
- Installable Android APK file or a reviewer-accessible APK download link.
- The project must pass TypeScript checking, linting, tests, and the Android build.

Submit all required items within 48 hours. Incomplete submissions should clearly identify what remains unfinished.